

Refatoração de Testes e Detecção de Test Smells

Serviço de gerenciamento de usuários - UserService

Nome completo: Pendente de informação

Matrícula: Pendente de informação

Disciplina: Teste de Software

Data: 07/10/2026

Versão técnica concluída. Preencher nome completo e matrícula antes da entrega.

Síntese da execução

Verificação	Resultado observado
Suíte original	4 aprovados; 1 ignorado
ESLint inicial	4 erros; 2 avisos
Suíte limpa	17 aprovados; nenhum ignorado
Validação conjunta	21 aprovados; 1 ignorado original
ESLint da suíte limpa e serviço	0 erros; 0 avisos

Repositório configurado no clone

<https://github.com/Victorgabrielcruz/test-smelly>

Suíte entregue: `__tests__/userService.clean.test.js`. O arquivo original permanece em `test/userService.smelly.test.js`, conforme a estrutura real do clone.

1. Análise dos Test Smells

A leitura manual foi realizada antes da refatoração. O diagnóstico abaixo considera a suíte original, preservada integralmente, e relaciona os problemas aos seus riscos.

1.1 Eager Test - criação e busca no mesmo cenário

O teste das linhas 18-31 cria um usuário, verifica seu ID, busca o cadastro e verifica nome e status. Há duas ações intercaladas com expectativas. A falha pode ocorrer em contratos diferentes, dificultando a localização do problema e a manutenção do teste. Na suíte limpa, a criação e a consulta têm casos independentes, com preparação, uma ação principal e verificações específicas.

1.2 Conditional Test Logic - desativação em for e if

Nas linhas 33-52, um laço percorre um usuário comum e um administrador. Um if/else escolhe quais expectativas executar. A leitura exige simular o fluxo; mudanças nos dados podem alterar as verificações executadas. A versão limpa separa usuário comum, administrador e ID inexistente. Também verifica que o usuário comum continua cadastrado e que o administrador permanece ativo.

1.3 Fragile Test - formatação exata do relatório

Nas linhas 54-64, a expectativa exige ID, nome e status em uma linha com ordem, vírgulas, espaços e quebra de linha exatos. Isso acopla o teste à apresentação: o relatório pode continuar correto e o teste falhar. A refatoração verifica os IDs e nomes dos usuários e os estados ativo e inativo, sem exigir a linha inteira ou o cabeçalho decorativo. Essa escolha pressupõe que o formato não é um contrato de integração externo, como sugere o comentário do serviço.

Problemas adicionais identificados

Exceção verificada somente no catch (linhas 66-74): se a validação de idade não lançar erro, nenhuma expectativa roda e o teste passa. Esse falso positivo foi corrigido com `toThrow` e é demonstrado na próxima página.

Teste ignorado e vazio (linhas 77-79): `test.skip` impede a execução e deixa o cenário sem proteção. O nome também menciona uma lista, embora a API produza texto. Foi implementado um caso ativo para a mensagem de ausência de usuários no relatório.

Isolamento e limites do escopo

O `beforeEach` instancia o serviço e usa `_clearDB()` para limpar o banco em memória. Esse mecanismo foi mantido. Não houve alteração em `src/userService.js`. O uso da API auxiliar de limpeza é uma dependência explícita do projeto; não caracteriza, por si só, um recurso externo oculto ou Convidado Misterioso.

2. Processo de refatoração

Foi escolhido o teste de menor de idade porque ele pode mascarar a ausência de uma regra de negócio. Os trechos abaixo reproduzem os testes; apenas os comentários explicativos do trecho original foram omitidos.

Antes - expectativa executada apenas se houver erro

```
test('deve falhar ao criar usuário menor de idade', () => {
  try {
    userService.createUser('Menor', 'menor@email.com', 17);
  } catch (e) {
    expect(e.message).toBe('O usuário deve ser maior de idade.');
```

Depois - verificação obrigatória da exceção, com AAA

```
test('deve rejeitar um usuário com 17 anos', () => {
  // Arrange
  const nome = 'Menor';
  const email = 'menor@teste.com';
  const idade = 17;

  // Act
  const criarUsuario = () => userService.createUser(nome, email, idade);

  // Assert
  expect(criarUsuario).toThrow('O usuário deve ser maior de idade.');
```

A ação é representada por uma função, pois `toThrow` precisa executar a chamada dentro do matcher para observar a exceção. Se o serviço aceitar 17 anos, a expectativa falha. A mensagem também é verificada, distinguindo a validação de idade de outros erros. Não há `try/catch` nem expectativas condicionais.

Decisões aplicadas aos demais cenários

A criação foi separada da busca; a desativação ganhou casos independentes; o relatório passou a verificar informações relevantes; o cenário vazio foi ativado. Os campos obrigatórios usam `test.each`: cada linha gera um teste independente, com o mesmo comportamento esperado e sem `if` ou `for` no corpo do teste. O limite de 18 anos e a preservação dos cadastros também foram verificados.

Experimentos controlados em cópias temporárias

Mudança simulada	Suíte original / suíte limpa
Remoção da validação de idade	Passa / falha: defeito detectado pela limpa
Alteração visual do relatório	Falha / passa: dados continuam preservados

Os experimentos não modificam o serviço real. Os resultados e saídas estão em `docs/evidencias/regressoes.json`. São duas verificações específicas, não uma campanha completa de testes de mutação.

3. Ferramenta, validação e conclusão

A configuração `.eslintrc.json` usa `eslint:recommended`, `plugin:jest/recommended` e as três regras solicitadas. ESLint 8.57.1 e `eslint-plugin-jest` 28.14.0 foram fixados para reproduzir o formato legado do enunciado. A suíte limpa também ativa `jest/no-conditional-in-test`.

Primeira execução - `npx eslint .`

```
ESLint - saída registrada na primeira execução

$ npx eslint .

C:\Users\vgppl\Documents\test-smelly\test\userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning Tests should not be skipped          jest/no-disabled-tests
 77:3  warning Test has no assertions          jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

Figura 1 - Captura do log real da primeira execução, exibido em um visualizador HTML para legibilidade. O texto original é entregue em `eslint-inicial.txt`.

A regra `no-conditional-expect` apontou três expectativas no `if/else` de desativação e uma no `catch`. Os avisos `no-disabled-tests` e `expect-expect` apontaram o teste ignorado e sem expectativas. A ferramenta não identificou automaticamente Eager Test nem a fragilidade semântica do relatório; esses itens vieram da revisão manual.

Validação final

A suíte limpa passou com 17 testes e o lint terminou com código 0, sem erros nem avisos. A execução conjunta aprovou 21 testes, com um único teste ignorado na suíte original. A cobertura da suíte limpa foi de 100% em linhas, instruções, funções e branches. O lint global ainda retorna 4 erros e 2 avisos no original preservado, resultado esperado para a comparação antes/depois.

Conclusão

Testes pequenos, nomes descritivos, AAA e expectativas obrigatórias tornam as falhas mais fáceis de interpretar e reduzem manutenção sem valor funcional. A análise estática oferece feedback repetível; a revisão manual avalia o significado e a fragilidade das verificações. Cobertura indica execução de código, mas não prova a detecção de todos os defeitos. Os experimentos confirmam duas melhorias concretas da suíte refatorada.

Referências: enunciado do trabalho; [documentação oficial do eslint-plugin-jest](#); [documentação oficial do ESLint 8](#). Evidências locais: `docs/evidencias/`.